

OWASP Top 10 (Version 2021)

Las 10 vulnerabilidades más críticas en aplicaciones web y sus mitigaciones en Spring Boot & Java.

Lautaro Olivero - Programación IV – TUP

INDICE

A01:2021 Broken Access Control — Control de Acceso Roto.

A02:2021 Cryptographic Failures — Fallos Criptográficos.

A03:2021 Injection — Inyección.

A04:2021 Insecure Design — Diseño Inseguro.

A05:2021 Configuración de Seguridad Incorrecta.

A06:2021 Componentes Vulnerables y Desactualizados.

A07:2021 Identification and Authentication Failures — Fallos de Identificación y Autenticación.

A08:2021 Software and Data Integrity Failures — Fallos de Integridad de Software y Datos

A09:2021 Security Logging and Monitoring Failures — Fallos de Registro y Monitoreo

A10:2021 Server-Side Request Forgery (SSRF) — Falsificación de Solicitudes del Lado del Servidor

Documentación oficial:

- OWASP Top 10 2021 — owasp.org/Top10
- Spring Security Reference Documentation — docs.spring.io
- Spring Boot Reference Documentation — docs.spring.io
- NVD (National Vulnerability Database) — nvd.nist.gov

A01:2021 Broken Access Control — Control de Acceso Roto.

- Definición Técnica

El control de acceso roto ocurre cuando los mecanismos que restringen lo que los usuarios autenticados pueden hacer fallan o no están correctamente implementados. Esto permite que un usuario acceda a recursos, funciones o datos a los que no debería tener permiso.

- Mecanismo de Ataque

- IDOR (Insecure Direct Object Reference): Modifica parámetros como `?id=123` a `?id=124` para acceder a datos de otro usuario.
- Escalada de privilegios vertical: Un usuario común accede a endpoints de administrador navegando directamente a `/admin/users`.
- Escalada horizontal: Accede a recursos de otro usuario con el mismo nivel de privilegios.
- Manipulación de JWT o tokens: Cambia el claim de rol para obtener permisos elevados

- Estrategias de Mitigación (Spring Boot & Java)

- Usar Spring Security con anotaciones `@PreAuthorize` y `@Secured` en métodos y controladores.
- Implementar verificación de pertenencia de recursos: validar que el recurso solicitado pertenezca al usuario autenticado.
- Configurar reglas de acceso por URL con `HttpSecurity` en la clase de configuración de seguridad.
- Aplicar principio de mínimo privilegio: denegar todo por defecto, permitir explícitamente.
- Usar `@PathVariable` con validación contra el contexto del usuario autenticado.

- Caso Real Conocido

Entidad: Facebook (Meta) — Año: 2018 Una vulnerabilidad en la función 'Ver como' (View As) permitió a atacantes obtener tokens de acceso de otros usuarios. El fallo combinaba

tres bugs distintos, uno de los cuales estaba relacionado con control de acceso incorrecto en la generación de tokens. Consecuencias: se vieron comprometidas aproximadamente 50 millones de cuentas. Los atacantes podían acceder a perfiles ajenos como si fueran el propio usuario. Facebook se vio obligado a cerrar sesión a 90 millones de usuarios como medida preventiva.

A02:2021 Cryptographic Failures — Fallos Criptográficos.

- Definición Técnica

Esta categoría engloba todos los fallos relacionados con el uso incorrecto o ausente de criptografía para proteger datos sensibles (contraseñas, datos de tarjetas, información personal, tokens). Incluye el uso de algoritmos obsoletos (MD5, SHA-1, DES), transmisión de datos sin cifrar, almacenamiento de contraseñas en texto plano o con hashes débiles, y claves criptográficas hardcodeadas en el código fuente.

- Mecanismo de Ataque

- Sniffing de red: Si los datos se transmiten en HTTP sin TLS, un atacante en la red puede capturarlos con Wireshark o herramientas similares.
- Rainbow tables: Si las contraseñas se almacenan con “MD5” o “SHA-1 sin salt”, el atacante que accede a la base de datos puede revertirlas en segundos usando tablas precomputadas.
- Exposición de backups: Archivos de configuración o backups de BD sin cifrar exponen credenciales.
- Claves hardcodeadas: Claves en el código fuente quedan expuestas en repositorios públicos.

- Estrategias de Mitigación (Spring Boot & Java)

- Usar BCrypt o Argon2 para el hash de contraseñas mediante PasswordEncoder de Spring Security.
- Forzar HTTPS/TLS 1.2+ en todas las comunicaciones con `server.ssl.*` en `application.properties`.
- Nunca almacenar datos sensibles sin cifrar. Usar Jasypt para cifrar `properties` sensibles.
- Externalizar claves y secretos en Spring Cloud Config, HashiCorp Vault o variables de entorno.

- Deshabilitar algoritmos obsoletos en la configuración de TLS/SSL

- Caso Real Conocido

Entidad: LinkedIn — Año: 2012 (revelado completamente en 2016) LinkedIn sufrió una brecha donde se robaron 117 millones de credenciales. Las contraseñas estaban hasheadas con SHA-1 sin salt, lo que permitió a los atacantes crackear la mayoría en poco tiempo usando rainbow tables. Consecuencias: millones de contraseñas expuestas, daño reputacional masivo, demandas legales y multas regulatorias. El mismo dataset se vendió en el mercado negro años después.

A03:2021 Injection — Inyección.

- Definición Técnica

La inyección ocurre cuando datos no confiables se envían a un intérprete como parte de un comando o consulta. El intérprete ejecuta el input del atacante como si fuera código legítimo. Los tipos más comunes son: **SQL Injection, NoSQL Injection, LDAP Injection, OS Command Injection y JNDI Injection.**

- Mecanismo de Ataque

Ejemplo de SQL Injection:

La aplicación construye la consulta concatenando input del usuario:

```
SELECT * FROM users WHERE user= [valor del usuario]
```

El atacante ingresa: ' OR '1'='1

Resultado: `SELECT * FROM users WHERE user="" OR '1'='1'`

Esto retorna todos los registros. Con variantes más agresivas puede eliminar tablas, extraer contraseñas o ejecutar comandos del sistema operativo si el usuario de BD tiene permisos suficientes.

- Estrategias de Mitigación (Spring Boot & Java)

- Usar siempre Prepared Statements / Parameterized Queries con JPA, Hibernate o Spring JDBC.

- Con Spring Data JPA, los métodos de repositorio y JPQL parametrizado son seguros por defecto.
- Validar y sanitizar todo input del usuario con Bean Validation (JSR-380) y anotaciones como @NotBlank, @Pattern, @Size.
- Usar ORMs correctamente (evitar queries nativas con concatenación de strings).
- Aplicar el principio de mínimo privilegio en el usuario de base de datos.

• Caso Real Conocido

Entidad: Equifax — Año: 2017

Aunque la vulnerabilidad inicial fue Apache Struts (CVE-2017-5638, relacionada con inyección de comandos OGNL), el ataque de Equifax incluyó componentes de inyección que permitieron a los atacantes moverse lateralmente dentro de la red.

Consecuencias: datos personales (SSN, fechas de nacimiento, direcciones) de 147 millones de personas comprometidos. Multa de 575 millones de dólares acordada con la FTC y graves daños reputacionales permanentes.

A04:2021 Insecure Design — Diseño Inseguro.

• Definición Técnica

El diseño inseguro es una categoría amplia que representa la ausencia o ineficacia de controles de seguridad en la etapa de diseño de la arquitectura del sistema. A diferencia de otras vulnerabilidades, no es un error de implementación sino una falla conceptual: el sistema fue diseñado sin considerar amenazas, sin modelado de amenazas (threat modeling) y sin principios de seguridad desde el inicio (Secure by Design).

• Mecanismo de Ataque

- Un sistema de recuperación de contraseña que usa preguntas de seguridad simples permite ataques de fuerza bruta o ingeniería social sin límite de intentos.
- Una API de e-commerce que permite aplicar múltiples cupones de descuento sin restricciones lógicas.
- Un flujo de pago que puede completarse omitiendo pasos (manipulando el estado del lado del cliente).
- Ausencia de rate limiting que permite enumeración de usuarios masiva.

• Estrategias de Mitigación (Spring Boot & Java)

- Adoptar Threat Modeling en la etapa de diseño (metodología STRIDE).
- Implementar rate limiting con librerías como Bucket4j integrada con Spring Boot.
- Aplicar Secure by Design: definir qué está permitido y rechazar todo lo demás.
- Usar Spring Security para validar flujos de negocio críticos con CSRF protection.
- Documentar y revisar los flujos de negocio con perspectiva de abuso (abuse cases).
- Implementar límites de intentos fallidos con Spring Security's AccountStatusUserDetailsChecker.

- **Caso Real Conocido**

Entidad: Instagram (Meta) — Año: 2019 Investigadores descubrieron que el sistema de recuperación de cuentas mediante PIN de 6 dígitos no limitaba correctamente los intentos desde diferentes IPs. Esto constituye un fallo de diseño: el flujo de recuperación no fue diseñado con restricciones de seguridad adecuadas.

Consecuencias: habría permitido tomar control de cualquier cuenta de Instagram mediante fuerza bruta del PIN. Meta pagó una recompensa de bug bounty de 30.000 dólares al investigador que lo reportó.

A05:2021 Configuración de Seguridad Incorrecta.

- **Definición Técnica**

Ocurre cuando los sistemas, frameworks, servidores o plataformas cloud están configurados de manera insegura: credenciales por defecto sin cambiar, funcionalidades innecesarias habilitadas, mensajes de error que revelan información interna, permisos excesivos en S3 buckets, endpoints de administración expuestos, o configuraciones de seguridad de frameworks no activadas. Es una de las vulnerabilidades más comunes precisamente porque requiere atención constante en cada capa del stack.

- **Mecanismo de Ataque**

1. El atacante escanea el endpoint /actuador de Spring Boot y encuentra métricas internas, variables de entorno con contraseñas, o puede llamar a /actuador/shutdown.
2. Accede a la consola H2 en producción (/h2-console) sin autenticación.
3. Los mensajes de error muestran stack traces con nombres de clases internas y versiones de librerías.
4. Permisos de bucket S3 configurados como público exponen archivos privados.
5. Credenciales por defecto de bases de datos (root/root) accesibles desde internet.

- Estrategias de Mitigación (Spring Boot & Java)

- Deshabilitar o proteger endpoints de Spring Actuator: exponer solo los necesarios y protegerlos con autenticación.
- Desactivar la consola H2 en producción (spring.h2.console.enabled=false).
- Configurar manejo de errores centralizado con @ControllerAdvice sin revelar detalles internos.
- Usar Spring Security Headers: X-Content-Type-Options, X-Frame-Options, CSP.
- Automatizar revisiones de configuración con herramientas como OWASP Dependency-Check

- Caso Real Conocido

Entidad: Capital One — Año: 2019 Un firewall de aplicación web (WAF) mal configurado en AWS permitió a la atacante explotar una vulnerabilidad SSRF (Server-Side Request Forgery). La configuración incorrecta del rol IAM le dio acceso a metadatos de EC2 con credenciales temporales. Consecuencias: datos de más de 100 millones de clientes comprometidos (números de seguro social, cuentas bancarias). Multa de 80 millones de dólares impuesta por reguladores. Daño reputacional significativo para el banco

A06:2021 Componentes Vulnerables y Desactualizados.

- Definición Técnica

Esta vulnerabilidad se produce cuando una aplicación utiliza librerías, frameworks, módulos del SO, servidores de aplicaciones u otros componentes con vulnerabilidades conocidas y publicadas como NVD (National Vulnerability Database). El riesgo existe tanto en dependencias directas como transitivas. Incluye también componentes que ya no reciben mantenimiento de seguridad.

- Mecanismo de Ataque

1. El atacante identifica la versión del framework o librería usada (a través de headers HTTP, mensajes de error o archivos expuestos).
2. Busca CVEs conocidos para esa versión en bases públicas (CVE Details, NVD, Exploit-DB).

3. Descarga o adapta un exploit público y lo ejecuta contra la aplicación

- Estrategias de Mitigación (Spring Boot & Java)

- Usar OWASP Dependency-Check integrado en Maven/Gradle para detectar CVEs en dependencias.
- Mantener actualizado el pom.xml y usar Dependabot (GitHub) para actualizaciones automáticas.
- Suscribirse a alertas de seguridad de Spring (spring.io/security) y de las librerías utilizadas.
- Utilizar el Spring Boot BOM (Bill of Materials) para gestionar versiones compatibles y seguras.
- Incluir el plugin en el pipeline CI/CD para bloquear builds con vulnerabilidades críticas.

- Caso Real Conocido

Entidad: Múltiples organizaciones — Año: 2021 (Log4Shell, CVE-2021-44228) La vulnerabilidad en Apache Log4j 2 afectó a miles de aplicaciones Java en todo el mundo, incluyendo Apple, Amazon, Microsoft, Cloudflare y gobiernos. Permitía ejecución remota de código (RCE) sin autenticación mediante una simple cadena JNDI en cualquier campo loggeado.

Consecuencias: fue considerada una de las vulnerabilidades más graves de la historia. Explotada por grupos de ransomware y actores estatales para robo de datos y acceso persistente en infraestructuras críticas.

A07:2021 Identification and Authentication Failures — Fallos de Identificación y Autenticación.

- Definición Técnica

Esta categoría engloba debilidades en los mecanismos de autenticación e identificación de usuarios: ausencia de autenticación multifactor (MFA), contraseñas débiles o por defecto permitidas, gestión insegura de sesiones (tokens que no expiran, IDs de sesión predecibles).

- Mecanismo de Ataque

- Credential Stuffing: El atacante usa listas de usuarios/contraseñas filtradas de otras brechas y las prueba automáticamente (muchos usuarios reutilizan contraseñas).
 - Brute Force: Sin limitación de intentos, prueba millones de combinaciones.
 - Session Fixation: Fuerza al usuario a usar un ID de sesión conocido por el atacante.
 - JWT débiles: Tokens con algoritmo 'none', secretos débiles o sin expiración permiten falsificar identidades.
 - Password en URL o logs: Contraseñas expuestas en logs del servidor.
- Estrategias de Mitigación (Spring Boot & Java)
 - Implementar MFA con TOTP usando librerías como Google Authenticator + JWT.
 - Configurar Account Lockout en Spring Security tras N intentos fallidos.
 - Usar Spring Session con Redis para gestión de sesiones distribuida y Segura.
 - Regenerar el ID de sesión tras login exitoso para prevenir Session Fixation.
 - Usar JWT con algoritmos robustos (RS256) y tiempos de expiración cortos
 - Validar contraseñas contra listas de contraseñas comunes (Have I Been Pwned API).

- Caso Real Conocido

Entidad: Dropbox — Año: 2012 (descubierto en 2016) Atacantes robaron un archivo de empleados de Dropbox que contenía hashes de contraseñas. Usando credenciales obtenidas de otra brecha (LinkedIn 2012), lograron acceder a una cuenta de empleado sin MFA. Desde allí obtuvieron 68 millones de registros de usuarios.

Consecuencias: **68 millones de combinaciones de email/contraseñas expuestas**, daño masivo a la reputación, implementación obligatoria de MFA y auditorías de seguridad exhaustivas.

A08:2021 Software and Data Integrity Failures — Fallos de Integridad de Software y Datos

- Definición Técnica

Esta categoría abarca fallos que permiten a un atacante comprometer la integridad del código o los datos de una aplicación. Incluye: pipelines CI/CD inseguros donde código malicioso puede inyectarse, actualizaciones de software sin verificación de firma/integridad, y dependencias descargadas de fuentes no verificadas.

- **Mecanismo de Ataque**

- Deserialización insegura: La aplicación Java deserializa objetos de fuentes no confiables. El atacante envía un payload serializado malicioso (ej. con ysoserial) que ejecuta comandos arbitrarios.
- Supply Chain Attack: El atacante compromete una librería de terceros (subiendo una versión maliciosa a npm/Maven Central) que es descargada automáticamente. 3.
- CI/CD poisoning: Acceso al repositorio o pipeline para inyectar código malicioso en el artefacto final.

- **Estrategias de Mitigación (Spring Boot & Java)**

- Evitar la deserialización de datos de fuentes no confiables. Preferir JSON/XML con parsers dedicados.
- Si se necesita deserializar, usar whitelisting de clases permitidas con ObjectInputFilter de Java 9+.
- Verificar la integridad de dependencias con checksums SHA-256 en Maven (plugin enforcer).
- Proteger el pipeline CI/CD: revisar código, usar secrets management, firmar artefactos.
- Implementar firma digital de JARs y verificarla en el proceso de despliegue.

- **Caso Real Conocido**

Entidad: SolarWinds — Año: 2020 En uno de los ataques a la cadena de suministro más sofisticados de la historia, actores estatales rusos (APT29) comprometieron el proceso de build de SolarWinds Orion e insertaron un backdoor (SUNBURST) en actualizaciones legítimas firmadas digitalmente.

Consecuencias: el malware se distribuyó a ~18.000 organizaciones clientes, incluyendo agencias del gobierno de EE.UU. (Treasury, Pentagon, DHS). Considerado uno de los ciberataques más graves de la historia reciente.

A09:2021 Security Logging and Monitoring Failures — Fallos de Registro y Monitoreo

- **Definición Técnica**

Esta categoría describe la ausencia o insuficiencia de logging de eventos de seguridad y monitoreo de la aplicación. Sin registros adecuados, los ataques no se detectan, no se pueden investigar y no se puede responder a tiempo.

- **Mecanismo de Ataque**

- Puede realizar reconocimiento prolongado sin ser detectado (días, semanas).
- Los ataques de fuerza bruta pasan desapercibidos sin alertas de múltiples intentos fallidos.
- Movimiento lateral dentro de la red sin detección.
- Exfiltración de datos durante semanas antes de que alguien note anomalías de tráfico.

- **Estrategias de Mitigación (Spring Boot & Java)**

- Usar SLF4J + Logback o Log4j2 con niveles apropiados. Nunca loggear datos sensibles.
- Integrar con Spring Security para auditar eventos: login exitoso/fallido, acceso denegado.
- Implementar Spring Boot Actuator con métricas y exportarlas a Prometheus + Grafana.
- Enviar logs a un sistema centralizado: ELK Stack (Elasticsearch, Logstash, Kibana) o Splunk.
- Configurar alertas automáticas para patrones sospechosos (N fallos en X segundos).

- **Caso Real Conocido**

Entidad: Uber — Año: 2016 (revelado en 2017) Atacantes accedieron a un repositorio privado de GitHub con credenciales de AWS, descargaron datos de 57 millones de usuarios y conductores. Uber no detectó la brecha durante meses por falta de monitoreo adecuado, y encima pagó 100.000 dólares a los atacantes para encubrirlo.

Consecuencias: 57 millones de afectados, multas regulatorias de 148 millones de dólares en EE.UU., investigaciones en múltiples países y renuncia del CISO.

A10:2021 Server-Side Request Forgery (SSRF) — Falsificación de Solicitudes del Lado del Servidor

- **Definición Técnica**

SSRF ocurre cuando una aplicación web obtiene un recurso remoto usando una URL proporcionada (directa o indirectamente) por el usuario, sin validar ni filtrar adecuadamente esa URL. El atacante puede hacer que el servidor realice peticiones HTTP a destinos arbitrarios, incluyendo recursos internos de la red privada a los que normalmente no tendría acceso desde internet. Esto permite acceder a servicios internos, metadata de cloud providers y hasta ejecutar código en sistemas internos.

- **Mecanismo de Ataque**

1. La aplicación ofrece funcionalidad de 'preview de URL' o 'importar desde enlace'.
2. El atacante proporciona: `http://169.254.169.254/latest/meta-data/iam/security-credentials/`
3. El servidor realiza la petición a la dirección de metadata de AWS (solo accesible internamente).
4. Obtiene credenciales temporales de IAM con las que puede acceder a S3, RDS, etc.

Otras URLs objetivo: `http://localhost:8080/admin`, `http://redis:6379`, servicios internos sin autenticación por estar 'protegidos' por la red interna.

- **Estrategias de Mitigación (Spring Boot & Java)**

- Validar y sanitizar todas las URLs proporcionadas por el usuario con una allowlist de dominios/IPs permitidos.
- Bloquear peticiones a rangos de IPs privadas (10.x.x.x, 172.16.x.x, 192.168.x.x, 127.x.x.x, 169.254.x.x) usando `InetAddress.isLoopbackAddress()` e `isSiteLocalAddress()`.
- En AWS, usar IMDSv2 que requiere token de sesión (previene SSRF directo).
- Usar `RestTemplate` o `WebClient` con interceptores que validen la URL de destino.
- Deshabilitar redirecciones automáticas HTTP en el cliente.

- **Caso Real Conocido**

Entidad: Capital One — Año: 2019 La atacante (ex-empleada de AWS) explotó una vulnerabilidad SSRF en el WAF de Capital One configurado sobre EC2. Envío una petición SSRF hacia el endpoint de metadata de AWS (169.254.169.254) y obtuvo credenciales IAM del rol asignado al servidor. Con esas credenciales accedió a más de 700 carpetas en S3.

Consecuencias: datos de 106 millones de solicitantes de tarjetas de crédito comprometidos, multa de 80 millones de dólares y pérdida de confianza masiva.